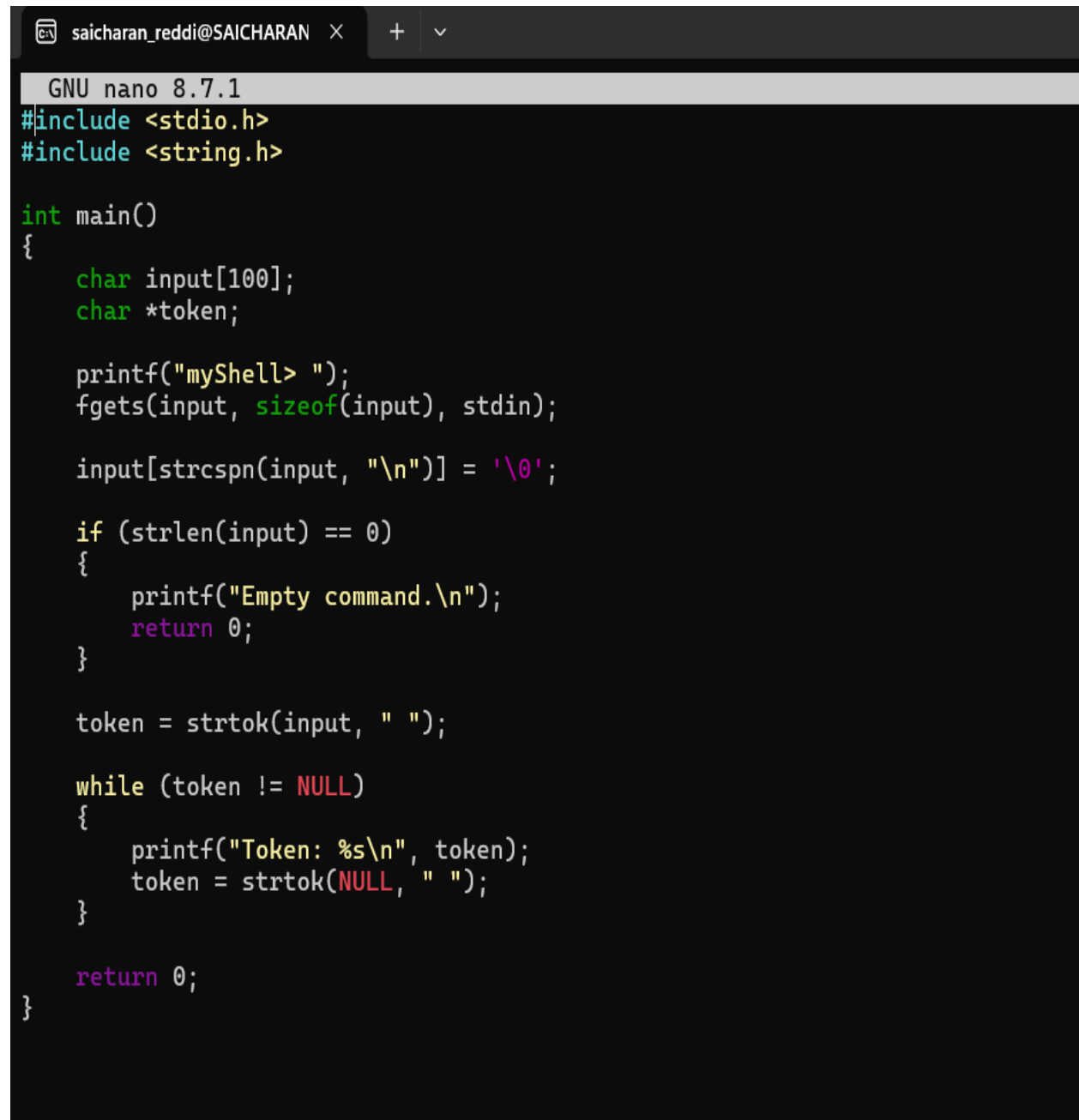


SKILL WEEK-4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE:

A screenshot of a terminal window with a dark background. The window title bar shows a browser icon, the username 'saicharan_reddi@SAICHARAN', and window control buttons. The terminal header indicates 'GNU nano 8.7.1'. The code is written in C and uses syntax highlighting: keywords are green, strings are yellow, and literals like NULL are red. The code implements a simple shell that reads a line of input, strips the trailing newline, and then tokenizes it using strtok. It prints each token on a new line. If the input is empty, it prints 'Empty command.\n' and returns 0. The code is enclosed in a main function with curly braces.

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    token = strtok(input, " ");

    while (token != NULL)
    {
        printf("Token: %s\n", token);
        token = strtok(NULL, " ");
    }

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARAN x + v
root@Mani: nano Parsing.c
root@Mani: gcc Parsing.c -o Parsing
root@Mani: ./Parsing
myShell> |
```

REPORT: The input was successfully divided into individual tokens using delimiters such as spaces and tabs. The program correctly displayed each argument separately and handled empty commands. Thus, the basic tokenization and parsing functionality required for the shell was successfully implemented.

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
saicharan_reddi@SAICHARAN x + v
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched single quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}|
```

OUTPUT:

```
saicharan_reddi@SAICHARAN x + v
root@Mani: nano singlequotes.c
root@Mani: gcc singlequotes.c -o singlequotes
root@Mani: ./singlequotes
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
root@Mani: |
```

REPORT: The Single-quoted strings were successfully detected and processed. The literal content inside the quotes was preserved, variable expansion was not performed, and unmatched single quotes were detected and reported as syntax errors.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
saicharan_reddi@SAICHARAN x + v
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '"')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched double quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}|
```

OUTPUT:

```
root@Mani:~# nano Doublequotes.c
root@Mani:~# gcc Doublequotes.c -o Doublequotes
root@Mani:~# ./Doublequotes
myShell> echo 'Hello World'
Parsing successful.
root@Mani:~#
```

REPORT: The Double-quoted input was successfully identified and parsed. Spaces inside quotes were preserved, variable expansion was supported conceptually, and unmatched double quotes were detected as syntax errors.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```
saicharan_reddi@SAICHARAN GNU nano 8.7.1
#include <stdio.h>

int main()
{
    char input[200];
    char output[200];
    int i, j = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;
            output[j++] = input[i];
        }
        else
        {
            output[j++] = input[i];
        }
    }

    output[j] = '\0';

    printf("Parsed Input: %s", output);

    return 0;
}
```

OUTPUT:

```
root@Mani:~# nano Escape.c
root@Mani:~# gcc Escape.c -o Escape
root@Mani:~# ./Escape
myShell> echo 'Hello World'
Parsed Input: echo 'Hello World'
root@Mani:~#
```

REPORT: Escape sequences were successfully handled. The program preserved escaped spaces and special characters and produced the corresponding parsed output.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

CODE:

```
GNU nano 8.7.1 Execute.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    printf("mysHELL$ ls\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        execlp("ls", "ls", NULL);

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        wait(&status);

        printf("Child exited with status: %d\n",
```

```

        printf("Child exited with status: %d\n",
               WEXITSTATUS(status));
    }

    return 0;
}

```

OUTPUT:

```

root@Mani:~# nano Execute.c
root@Mani:~# gcc Execute.c -o Execute
root@Mani:~# ./Execute
myshell$ ls
Parent Process PID: 828
Child Process PID: 829
Created Child PID: 829
BufferSize.c.save  Escape      ForkProgram.c  PROCESSTASK.save  SystemCall      command_executor  filecopy.c      process.c      snap
ChildProcess       Escape.c      LICENSE        ParentChild       SystemCall.c     command_executor.c  fliecopy.c.save  scripts       src
ChildProcess.c     Execute      Makefile       ParentChild.c     SystemCallParent  commandhistory     include          shell         tests
Command.c          Execute.c    MemoryList     Parsing           SystemCallParent.c  commandhistory.c  interactiveLoop  shell.c
DisplayPID         File1.txt   MemoryList.c  Parsing.c         assets           copyfile.c        interactiveLoop.c  shellcommand
DisplayPID.c       File2.txt   OSSP          ProcessParent     bin              docs              kill.c           shellcommand.c
DoubleQuotes       FileManagement  OSSP_old      ProcessParent.c  command          file              obj             singlequotes
DoubleQuotes.c     ForkProgram  OS_LAB        README.md         command.c        file.c            process          singlequotes.c
Child exited with status: 0
root@Mani:~#

```

REPORT: The child process was successfully created and used to execute a command. The parent process successfully waited for the child and obtained its exit status. Execution errors were also handled.